Relaciones y colaboración entre objetos

Ing. Andrés Eduardo Paternina Ariza Esp.
Universidad del Magdalena
apaternina@unimagdalena.edu.co

¿Dónde estábamos?

En las sesiones anteriores construimos:

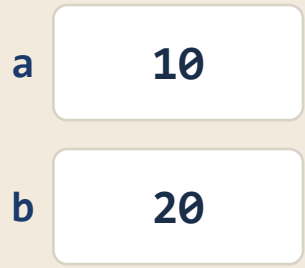
- **Definición de Clase y Objeto**
- **Abstracción**
- **Encapsulamiento**
- **Herencia**

¿Qué ocurre al asignar una variable a otra?

```
int a = 10;
int b = a;
b = 20;

Console.WriteLine(a);
Console.WriteLine(b);
```

En memoria



Pregunta para el grupo

¿Qué imprime?

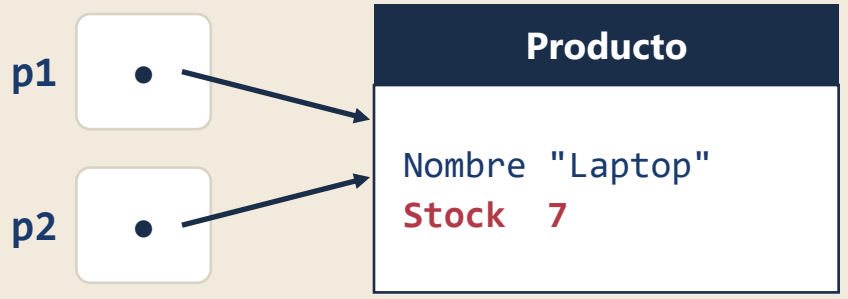
¿Se modificó a?

¿Qué ocurre con dos referencias al mismo objeto?

```
Producto p1 = new Producto("Laptop", 10);
Producto p2 = p1;
p2.DescontarStock(3);

Console.WriteLine(p1.Stock);
Console.WriteLine(p2.Stock);
```

En memoria



Pregunta para el grupo

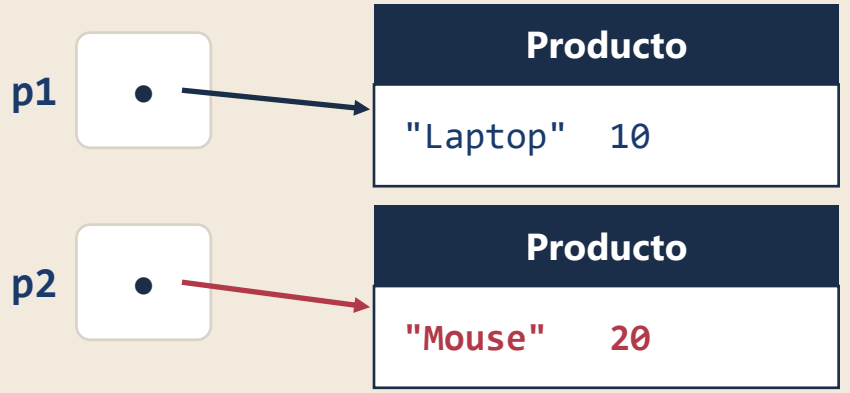
¿Hay uno o dos objetos Producto?

Modificar un objeto no es reasignar una variable

```
Producto p1 = new Producto("Laptop", 10);
Producto p2 = p1;
p2 = new Producto("Mouse", 20);

Console.WriteLine(p1.Nombre);
Console.WriteLine(p2.Nombre);
```

En memoria



Pregunta para el grupo

¿Cuántos objetos existen después del segundo new?

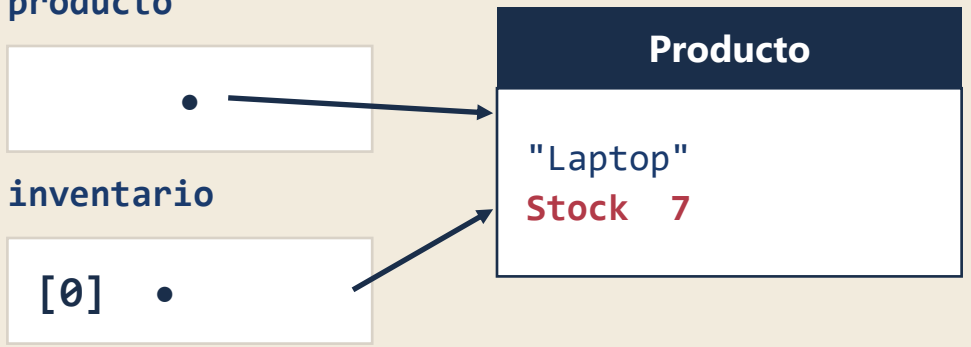
¿Qué sucede al guardar objetos en una lista?

```
Producto producto =
    new Producto("Laptop", 10);
List<Producto> inventario = new();

inventario.Add(producto);
producto.DescontarStock(3);

Console.WriteLine(inventario[0].Stock); // 7
```

En memoria



Pregunta para el grupo

¿Qué imprime?

¿Y si añadimos un producto creado con otro new?

Sistema de vehículos

Una empresa desea desarrollar una primera versión de su sistema para controlar el funcionamiento de sus vehículos. De cada motor se almacena número de serie, potencia en caballos de fuerza y estado de funcionamiento. Todo motor inicia apagado. Un motor permite encenderse y apagarse. De cada vehículo se almacena placa, marca y el motor que tiene asignado. Cada vehículo tiene exactamente un motor desde que se registra. Un vehículo permite encenderse, apagarse y consultar si se encuentra en funcionamiento. Estas acciones dependen de su motor: un vehículo está en funcionamiento solo si su motor está encendido.

Cada vehículo tiene un único motor

Vehiculo

Placa, marca y su motor asignado

tiene un

1 — 1

Motor

Número de serie, potencia y estado

```
class Vehiculo
{
    Motor motor;   // un único motor
}
```

Cómo colaboran

1. El programa llama a vehiculo.Encender()
2. Vehiculo delega en motor.Encender()
3. Motor revisa su estado y lo cambia

Motor controla su propio estado

```
public class Motor
{
    public string NumeroSerie { get; private set; }
    public int Potencia { get; private set; }
    public bool Encendido { get; private set; }

    public Motor(string numeroSerie, int potencia)
    {
        NumeroSerie = numeroSerie;
        Potencia = potencia;
        Encendido = false;
    }

    public void Encender()
    {
        if (Encendido) return;
        Encendido = true;
    }

    public void Apagar()
    {
        if (!Encendido) return;
        Encendido = false;
    }
}
```

Qué cumple del enunciado

- Conoce número de serie, potencia y estado
- Solo Motor cambia Encendido: el set es privado
- Encender y Apagar no repiten la operación si ya está en ese estado

EJERCICIO 1 · SOLUCIÓN

Vehiculo delega el encendido en su motor

```
public class Vehiculo
{
    public string Placa { get; private set; }
    public string Marca { get; private set; }
    private Motor Motor;
    public Vehiculo(string placa, string marca, Motor motor)
    {
        Placa = placa;
        Marca = marca;
        Motor = motor;
    }

    public void Encender() { Motor.Encender(); }
    public void Apagar() { Motor.Apagar(); }

    public bool EstaEnFuncionamiento() { return Motor.Encendido; }
}
```

Qué cumple del enunciado

- Conoce placa, marca y un único motor, recibido al crearse
- Encender y Apagar le piden la operación al motor
- El estado se consulta al motor; Vehiculo no lo duplica ni lo modifica

Así colaboran Vehiculo y Motor

```
Motor motor = new Motor("MT-2041", 150);
Vehiculo carro = new Vehiculo("ABC123", "Mazda", motor);

carro.Encender();
Console.WriteLine(carro.EstaEnFuncionamiento());

carro.Apagar();
Console.WriteLine(carro.EstaEnFuncionamiento());
```

Pregunta para el grupo

¿Qué imprime la última línea?

Sistema de biblioteca

Una biblioteca necesita desarrollar un sistema que permita administrar sus libros y controlar su disponibilidad.

De cada libro se conoce su código, título, autor y estado de disponibilidad. Un libro puede prestarse o devolverse, siempre que su estado permita realizar la operación.

De cada biblioteca se conoce su nombre y el conjunto de libros que administra.

La biblioteca permite registrar nuevos libros, consultar cuántos tiene disponibles y realizar préstamos o devoluciones a partir del código de cada libro.

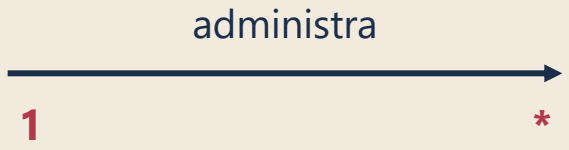
Cuando se solicita un préstamo, la biblioteca debe localizar el libro correspondiente y solicitarle que cambie su estado. No debe modificar directamente los atributos internos del libro.

Se requiere implementar las clases Libro y Biblioteca, estableciendo las relaciones necesarias para que sus objetos colaboren entre sí.

Cada biblioteca administra muchos libros

Biblioteca

Nombre y los libros que administra



Libro

Código, título, autor y disponibilidad

```
class Biblioteca
{
    List<Libro> libros;   // muchos libros
}
```

Cómo colaboran

1. Se llama a biblioteca.Prestar(codigo)
2. Biblioteca busca el libro por su código
3. Libro revisa su estado y lo cambia

Libro controla su propia disponibilidad

```
public class Libro
{
    public string Codigo { get; private set; }
    public string Titulo { get; private set; }
    public string Autor { get; private set; }
    public bool Disponible { get; private set; }

    public Libro(string codigo, string titulo, string autor)
    {
        Codigo = codigo;
        Titulo = titulo;
        Autor = autor;
        Disponible = true;
    }

    public void Prestar()
    {
        if (!Disponible) return;
        Disponible = false;
    }

    public void Devolver()
    {
        if (Disponible) return;
        Disponible = true;
    }
}
```

Qué cumple del enunciado

- Conoce código, título, autor y disponibilidad
- Solo Libro cambia Disponible: el set es privado
- Prestar y Devolver no hacen nada si el estado no lo permite

Biblioteca delega el préstamo en cada libro

```
public class Biblioteca
{
    public string Nombre { get; private set; }
    private readonly List<Libro> libros = new();

    public Biblioteca(string nombre) { Nombre = nombre; }
    public void RegistrarLibro(Libro libro) { libros.Add(libro); }
    public int ContarDisponibles() { return libros.Count(l => l.Disponible); }

    public void Prestar(string codigo)
    {
        Libro? libro = Buscar(codigo);
        if (libro == null) Console.WriteLine("El libro no existe");
        else libro.Prestar();
    }

    public void Devolver(string codigo)
    {
        Libro? libro = Buscar(codigo);
        if (libro == null) Console.WriteLine("El libro no existe");
        else libro.Devolver();
    }

    private Libro? Buscar(string codigo)
    {
        return libros.FirstOrDefault(l => l.Codigo == codigo);
    }
}
```

Qué cumple del enunciado

- Conoce su nombre y la lista de libros que administra
- Prestar y Devolver localizan el libro por código y le piden el cambio
- Nunca asigna Disponible: solo lo lee para contar

Así colaboran Biblioteca y Libro

```
Biblioteca bib = new Biblioteca("Central");
bib.RegistrarLibro(new Libro("L-01", "Rayuela", "Cortázar"));
bib.RegistrarLibro(new Libro("L-02", "Ficciones", "Borges"));

bib.Prestar("L-01");
Console.WriteLine(bib.ContarDisponibles());

bib.Prestar("L-01");
bib.Devolver("L-01");
Console.WriteLine(bib.ContarDisponibles());
```

Pregunta para el grupo

¿Qué imprime cada WriteLine?

¿Qué pasa con el segundo Prestar("L-01")?

Taller: gestión de habitaciones de hotel

Un hotel necesita desarrollar un sistema que permita administrar sus habitaciones y controlar los dispositivos instalados en ellas.

De cada aire acondicionado se conoce su número de serie, temperatura configurada y estado de funcionamiento (encendido o apagado). Un aire acondicionado permite encenderse, apagarse y modificar su temperatura, siempre que se encuentre encendido. La temperatura permitida debe estar entre 16 °C y 30 °C.

Cada habitación tiene un número, una capacidad máxima de huéspedes y un aire acondicionado asignado. Una habitación permite encender y apagar su aire acondicionado, ajustar su temperatura y consultar si se encuentra encendido. Cada habitación dispone de un único aire acondicionado.

El hotel tiene un nombre y administra un conjunto de habitaciones. Debe permitir registrar nuevas habitaciones, buscar una habitación por su número y consultar cuántas habitaciones tienen el aire acondicionado encendido.

Condiciones que debe cumplir el sistema

- No pueden existir dos habitaciones con el mismo número dentro de un hotel.
- La capacidad máxima de una habitación debe ser mayor que cero.
- Una habitación no puede existir en el sistema sin un aire acondicionado asignado.
- El número de serie de un aire acondicionado no puede modificarse después de su creación.
- No se puede cambiar la temperatura de un aire acondicionado apagado.
- Las operaciones sobre el aire acondicionado de una habitación deben realizarse a través de los comportamientos que ofrece la propia habitación.
- El hotel no debe modificar directamente los atributos internos de las habitaciones ni de sus aires acondicionados.

Actividad del taller

Desarrolle una aplicación de consola en C# que permita representar el escenario anterior. Identifique las clases necesarias, sus atributos, constructores, propiedades y métodos. Establezca las relaciones correspondientes y aplique encapsulamiento para garantizar el cumplimiento de las reglas.

Finalmente, desde el programa principal:

1. Cree un hotel con tres habitaciones, cada una con su respectivo aire acondicionado.
2. Registre las habitaciones en el hotel.
3. Encienda el aire acondicionado de dos habitaciones y configure sus temperaturas.
4. Intente cambiar la temperatura de un aire acondicionado que permanece apagado.
5. Consulte cuántas habitaciones del hotel tienen el aire acondicionado encendido.
6. Busque una habitación por su número y consulte el estado de su aire acondicionado.

Resultado esperado: una aplicación donde los objetos colaboren para realizar las operaciones solicitadas, sin acceder directamente a los atributos privados de otros objetos.

Para llevar

En POO no basta con almacenar datos de otros objetos.

Debemos decidir qué objetos conocen a cuáles, cómo se comunican y quién es responsable de cada regla.